

BottomScript

OFFICIAL REFERENCE BOOK & LANGUAGE MANUAL

A comprehensive guide covering complete syntax, immutable box data types, built-in features, operators, control flow, and runtime architecture for BottomScript v0.2.0.

VERSION 0.2.0 • AUTHORIZED GUIDE

Chapter 1: Introduction to BottomScript

BottomScript is a lightweight, custom-built interpreted programming language designed on top of Python. It features a clean syntax tailored for rapid script execution, immutable data safety using box wrappers, and robust control structures.

Execution Model

BottomScript source files use the `.bs` extension and can be executed through the CLI interpreter or compiled into standalone binary executables via PyInstaller.

```
# Run script using interpreter
python cLi.py script.bs

# Run using compiled binary
bottomscript.exe script.bs
```

Chapter 2: Comprehensive Data Types

BottomScript supports a rich set of native data types designed to handle numbers, textual strings, sequential lists, associative maps, and strict immutable boxes.

Data Type	Category	Description & Example
Number	Primitive	Integer and floating-point values (e.g., <code>let x = 10</code> , <code>let pi = 3.14</code>).
String	Primitive	Textual data enclosed in double quotes (e.g., <code>let name = "Tejaswi"</code>).
Array	Collection	Ordered sequence of elements (e.g., <code>let nums = [1, 2, 3, 4]</code>).
Dictionary	Collection	Key-value associative pairs (e.g., <code>let user = {"role": "Engineer"}</code>).
Box (Constant)	Immutable Wrapper	Locked constant container utilizing <code>:(...):</code> syntax (e.g., <code>let pi = :(3.14):</code>).

The Box Data Type (Immutable Constants)

The unique Box wrapper ensures that sensitive configuration parameters or mathematical constants cannot be accidentally reassigned during runtime execution.

```
let pi_val = :(3.14):
show pi_val

# Attempting re-assignment will throw a PermissionError:
# pi_val = 3.15 -> Error!
```

Chapter 3: Variables, Input & Output

Interacting with user terminals and managing variable states forms the core of BottomScript script execution.

Output Handling (`show``)

The `show` keyword prints evaluation results or expressions directly to the standard output console followed by a newline.

```
show "Welcome to BottomScript"
let total = 50 + 25
show total
```

Dynamic User Input (`take`)

The built-in `take` function prompts the user for interactive terminal inputs with optional prompt messages.

```
let username = take("Enter your name: ")
show "Hello, " + username
```

Chapter 4: Control Flow & Conditionals

Decision making is handled using structured conditional statements supporting full branching logic.

```
let score = 88

if (score >= 90) {
  show "Grade: A"
} elseif (score >= 75) {
  show "Grade: B"
} else {
  show "Grade: C"
}
```

Chapter 5: Tree Loops & Iteration

BottomScript utilizes the versatile `tree` keyword for handling all loop constructs, ranging from indexed numeric ranges to collections and dictionaries.

1. Range Loops

```
tree i in range(1, 4):
  show i
```

2. Array Iteration

```
let colors = ["Red", "Green", "Blue"]
tree c in colors:
    show c
```

3. Dictionary Key-Value Unpacking

```
let config = {"host": "localhost", "port": "8080"}
tree k, v in config:
    show k + ": " + v
```

Chapter 6: Functions & Modularity

Encapsulating reusable business logic is achieved using the `func` keyword, supporting parameters and explicit return signaling.

```
func calculateArea(length, width):
    let area = length * width
    return area

let result = calculateArea(10, 5)
show "Calculated Area:"
show result
```